

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

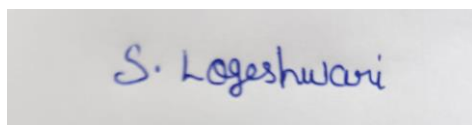
Total Marks: 50

Annexure A

Code of Conduct:

I Logeshwari S (192565040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. Secure Password Storage in Web Applications

A. Why storing plaintext passwords is insecure

Storing passwords in plaintext means that the actual passwords are directly saved in the application's database. If an attacker gains unauthorized access to the database, all user passwords can be immediately exposed.

This is dangerous because:

- Attackers can directly use the stolen passwords.
- Users often reuse the same password on multiple websites.
- Database administrators or unauthorized employees could potentially see the passwords.
- A single database breach can compromise many user accounts.

Therefore, passwords should **never be stored as plaintext**.

B. Secure method for storing passwords

A secure approach is to use a **password-hashing algorithm designed specifically for passwords**, such as **Argon2id**, with a unique random salt for every password. Alternatives include bcrypt or scrypt.

The process is:

~~Passwd~~ ~~Argon2id~~ ~~Passwd~~ $\rightarrow$ **Passwd-Argon2id-Password**

The database stores:

- The salt
- The resulting password hash
- The algorithm parameters

It does **not** store the original password.

When the user logs in, the application hashes the entered password using the stored salt and parameters and compares the resulting hash with the stored hash.

Justification: Password hashing is intentionally computationally expensive, making large-scale password guessing much more difficult after a database breach. A unique salt also prevents identical passwords from having identical stored hashes and helps defend against precomputed rainbow-table attacks.

Conclusion: Argon2id with a unique salt is a strong choice for secure password storage.

2. Digital Signature for Document Authentication

A. How digital signatures can be used

A company can use a **digital signature** to prove that an electronic document was issued by an authorized person and has not been modified.

The process is:

1. The sender creates the document.
2. A cryptographic hash of the document is generated.
3. The sender signs the hash using their **private key**.
4. The document and digital signature are sent to the recipient.
5. The recipient uses the sender's **public key** to verify the signature.
6. The recipient also calculates a hash of the received document.
7. If the values correspond, the signature is valid and the document has not been altered.

Conceptually:

~~Document~~ ~~Signature~~

The recipient verifies using:

~~Signature~~ ~~Document~~

B. Integrity and non-repudiation

Integrity:

A digital signature is linked to the document's cryptographic hash. If even a small part of the document is changed, its hash changes and signature verification fails.

Non-repudiation:

The signature is created using the sender's private key. Assuming the private key is securely controlled, the sender cannot reasonably deny having signed the document.

Digital signatures therefore provide:

- **Authentication** of the signer
- **Integrity** of the document
- **Non-repudiation** of the signed content

Conclusion: Digital signatures are suitable for contracts, certificates, invoices, official letters, and other electronically transmitted documents requiring authenticity.

3. Preventing Replay Attacks in Online Transactions

A. How replay attacks occur

A replay attack occurs when an attacker captures a legitimate transaction and later sends the same transaction again.

For example:

1. A customer sends a transaction requesting a payment of ₹5,000.
2. An attacker captures the authenticated transaction.
3. The attacker does not necessarily need to understand or decrypt it.
4. The attacker resends the captured transaction.
5. If the system accepts it as a new transaction, the payment may be processed again.

This can result in:

- Duplicate payments
- Unauthorized transactions
- Financial loss

The important point is that the attacker is **reusing a previously valid message**.

B. Cryptographic technique to prevent replay attacks

A suitable solution is to use a **unique nonce (number used once)** or transaction counter along with a cryptographic authentication mechanism such as an **HMAC**.

For example:

~~Transaction~~ Nonce-HMAC

The server maintains a record of used nonces or transaction identifiers.

Suppose a transaction contains:

- Transaction ID
- Amount
- Account information
- Unique nonce
- HMAC

When the server receives it, it:

1. Verifies the HMAC.
2. Checks whether the nonce/transaction ID has already been used.
3. Rejects the transaction if it has already been processed.

Therefore, replaying the old transaction will fail because its nonce has already been consumed.

Another common technique is using a **timestamp plus a nonce**, with the server accepting requests only within a short validity period.

Conclusion: A unique nonce combined with HMAC provides message authentication while ensuring that an old valid transaction cannot simply be reused.

4. Secure Communication over Public Wi-Fi

A. Risks involved

Using public Wi-Fi without encryption exposes users to several security risks.

1. Eavesdropping

An attacker on the same network may capture network traffic and potentially obtain sensitive information if the communication is not encrypted.

2. Man-in-the-Middle attack

An attacker may position themselves between the user and the intended server and attempt to intercept or modify communication.

3. Credential theft

If login credentials are transmitted without adequate encryption, attackers may potentially capture them.

4. Session hijacking

An attacker may attempt to obtain authentication/session information and impersonate the user.

5. Malicious or fake Wi-Fi networks

Attackers can create networks with names similar to legitimate public Wi-Fi networks and trick users into connecting.

B. Recommended secure communication method

The user should use **HTTPS/TLS** when accessing websites and applications containing sensitive information.

TLS provides:

- **Encryption** – protects data from network eavesdroppers.
- **Authentication** – certificates help the client verify the server's identity.

- **Integrity** – cryptographic authentication detects unauthorized modification of transmitted data.

For example:

$$U \xrightarrow{\text{TLS}} S$$

For additional protection on an untrusted network, a reputable **VPN** can provide an encrypted tunnel between the user's device and the VPN server. However, HTTPS/TLS remains essential because the VPN does not replace end-to-end application-layer security.

Conclusion: Sensitive information should be transmitted over properly configured HTTPS/TLS connections, especially when using public Wi-Fi.

5. Data Integrity in Cloud Storage

A. Using hashing to verify data integrity

A cryptographic hash function converts data into a fixed-length hash value.

For example:

$$Data \rightarrow SHA-256 \rightarrow Hash$$

Suppose an organization uploads a file to cloud storage.

Before uploading

The organization calculates:

$$(H_1)$$

The original hash is securely stored.

When verifying the data

The organization downloads the file and calculates:

$$(H_2)$$

Then:

$$H_1 = H_2$$

means that, with the usual assumptions of the hash function, the data has not been altered.

If:

$$H_1 \neq H_2$$

the file has changed or become corrupted.

Even a small change in the original data should produce a different cryptographic hash.

B. Additional mechanism to enhance trust

Hashing alone does not necessarily prove **who generated or approved the hash**. An attacker who can modify both the file and its stored hash could potentially defeat a simple comparison.

Therefore, an additional mechanism such as a **digital signature** should be used.

The organization can:

1. Calculate the hash of the data.
2. Sign the hash using its private key.
3. Store the data and signature in the cloud.
4. Later retrieve the data.
5. Recalculate the hash.
6. Verify the signature using the organization's public key.

Conceptually:

~~Data~~ ~~Signature~~

This provides stronger assurance because:

- **Hashing** detects changes to the data.
- **Digital signatures** provide integrity plus authentication of the signer.
- The private key prevents unauthorized parties from generating a valid organizational signature, assuming the key remains secure.

For especially sensitive cloud data, additional controls such as access control, audit logs, versioning, and authenticated encryption can also strengthen protection.